Java Pokemon Battle



Sapienza - Metodologie di Programmazione 1015884
Canale A-L 23-24 Prof. Mattia Samory

Lorenzo Cipolletti

Roma 09/06/2024

Features Implementate

Specifiche comuni:

Minimo:

- Implementati Bulbasaur, Charmander, Squirtle;
- Implementate statistiche base dei pokémon;
- Assegnate le mosse base a tutti i pokémon implementati;
- Adottato JavaFX 22 per il set grafico.

Tipico:

- Implementate meccaniche inerenti ai tipi delle mosse e dei pokémon con le loro statistiche.

Extra:

- Set crescita:
  - Level up;
  - IV;
  - EV (punti allenamento).
- Set multimedia:
  - Audio clip che accompagnano l'utente durante tutta l'esperienza;
  - Animazioni generiche: "barra della vita, inizio battaglia…";
  - Sprite personalizzati.
- Set programmatore: (in parte)
  - Jar eseguibile nella cartella out.

Sviluppatore singolo:

Minimo:

- Il giocatore affronta avversari fino alla sua prima sconfitta.

Tipico:

- Lo stato dei pokémon è preservato durante tutte le lotte;
- Implementata due schermate leaderboard, una accessibile dal menu e l'altra a fine partita.

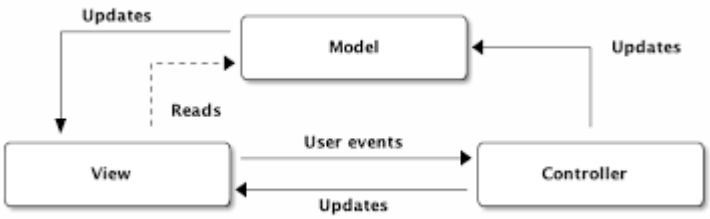
Extra:

- Implementato un semplice set botness che fornisce due diverse difficoltà.
  (facilmente estendibile)

Decisioni di Progettazione

Pattern MCV

In seguito ad una prima stesura delle classi principali del gioco e di alcune view, ho deciso di riscrivere il progetto seguendo un pattern architetturale MCV (modello-vista-controllo), al fine di dividere efficientemente la parte grafica da quella logica e potermi dedicare alle strutture principali del progetto.



Singletons

Durante una prima fase di design delle classi principali ho deciso di utilizzare un pattern singleton per tutte quelle classi che offrivano un servizio universale. Le classi che sfruttano questo pattern sono:

- **PokeService** (raccoglie i dati dei pokémon supportati)
- **MoveService** (raccoglie i dati delle mosse supportate)
- **Leaderboard** (raccoglie i dieci migliori score dei giocatori)
- **SoundService** (gestione audio)
- **GameState** (informazioni sullo stato attuale della partita)

```
    registerInfo(PokeEnum.BULBASAUR, new Bulbasaur());
    registerInfo(PokeEnum.IVYSAUR, new Ivysaur());
    registerInfo(PokeEnum.VENUSAUR, new Venusaur());
    registerInfo(PokeEnum.CHARMANDER, new Charmander());
    registerInfo(PokeEnum.CHARMELEON, new Charmeleon());
    registerInfo(PokeEnum.CHARIZARD, new Charizard());
    registerInfo(PokeEnum.SQUIRTLE, new Squirtle());
    registerInfo(PokeEnum.WARTORTLE, new Wartortle());
    registerInfo(PokeEnum.BLASTOISE, new Blastoise());
```

Principi applicati SRP, LSP, OCP e DIP

1) Ogni classe ha una sola ragione.
2) Di base ogni sottoclasse può essere sostituita dalla sua classe base o parent.
3) Classi come il CPUHandler, il PokemonInfo e la BaseMove sono facilmente estendibili senza modifiche.
4) Le Entità devono dipendere da astrazioni e non su classi concrete.
   Es: BaseMove, PokemonInfo, CPUHandler, i controller si interfacciano con le classi astratte o con le interfacce.

Design di composizioni e aggregazioni

```java
public class Pokemon implements PokemonInterface {
    private PokeMove[] moves = new PokeMove[4];

    private PokeStats iv = new PokeStats(15);
    private PokeStats ev = new PokeStats(255);
    private PokeStats effects = new PokeStats(6);

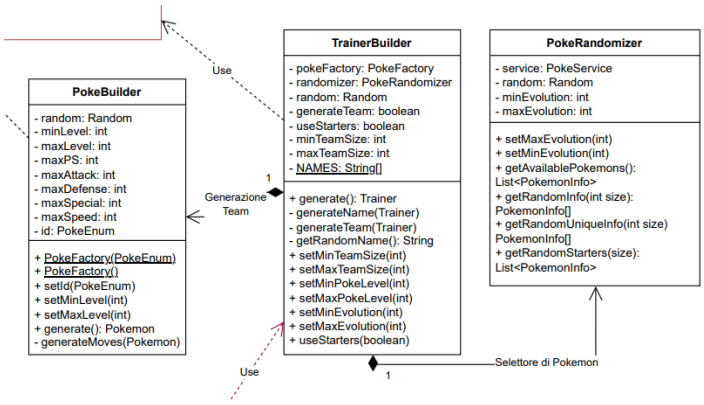
    private int level = 0;
    private int exp = 0;
    private int health = 0;

    // pokemon info
    private PokemonInfo info;
```

Builder Pattern

Per istanziare un Pokémon ci sono diversi passaggi da seguire, tra i quali l'assegnazione delle mosse, la definizione del tipo di Pokémon e l'impostazione delle statistiche personali. Nel caso delle battaglie Pokémon, è necessario fornire al giocatore uno starter iniziale e generare casualmente Pokémon e allenatori da far affrontare al giocatore.

Per raggiungere questo obiettivo, ho progettato la classe **PokemonBuilder**, in grado di generare un'istanza di Pokémon. Questo builder è poi utilizzato dal **TrainerBuilder** come componente aggiuntivo, così si evitano istanziazioni con argomenti non chiari o stati intermedi non validi.



Componenti grafiche personalizzate

Nella fase di design delle view, ho riscontrato la necessità di riutilizzare il codice esistente e di separare le logiche o i componenti in classi appropriate. Infatti, ho ideato una serie di classi ereditarie che estendono le classi base di JavaFX per realizzare facilmente componenti complesse.

Esempi:

GridMenu:



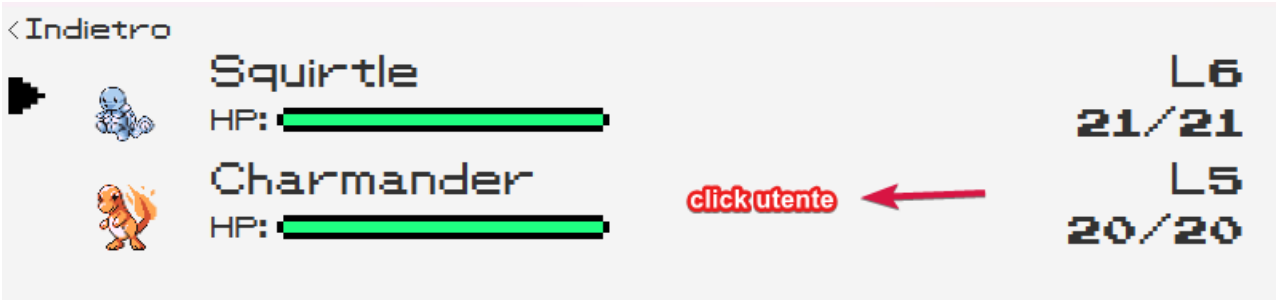
PokemonView, **PokemonInfoCard**, **HealthBar**



Callback pattern

Implementando diversi controller è sorta la necessità di eseguire determinate operazioni in seguito al compimento di certi eventi. Per questo motivo ho utilizzato il **Callback** pattern, un esempio di applicazione di questo pattern oltre nel **GridMenu** o in molte altre componenti UI lo troviamo nella classe **PokeTeamList**.

- Input utente



- Gestione input ed esecuzione callback per il cambio pokemon



Template method

Ho utilizzato il pattern comportamentale Template per le classi **PokemonInfo**, **BaseMove**, **TurnHandler**, **CPUInterface** … Questo approccio ha costretto le classi concrete a fornire le implementazioni necessarie, condividendo al contempo codice generico senza causare ripetizioni non necessarie.

```
5
6  public class Ember extends BaseMove {
7
8      public Ember(int requiredLevel) {
9          super(requiredLevel);
10     }
11
12     @Override
13     public String getName() {
14         return "Ember";
15     }
16
17     @Override
18     public MoveType getType() {
19         return MoveType.SPECIAL;
20     }
21
22     @Override
23     public PokeElement getElement() {
24         return PokeElement.FIRE;
25     }
26
27     @Override
28     public int getAccuracy() {
29         return 100;
30     }
31
32     @Override
33     public int getPower() {
34         return 40;
35     }
36
```

Risorse di riferimento per le meccaniche pokémon

- **Direttive progetto**
- Damage - Bulbapedia
- Stat - Bulbapedia
- Experience - Bulbapedia

Strumenti utili

- JavaFX Scene Builder (utile per ideare i layout in fxml)

Osservazioni

Per rendere l'esperienza di gioco più interessante e colmare la mancanza di cure nel gioco ho deciso di permettere ai pokémon del giocatore di curarsi tramite level up. Questa opzione è facilmente rimuovibile togliendo la chiamata **restore** dentro il metodo **levelUp** della classe **Pokemon**.

Durante le prime fasi del progetto non ho commentato tutto il codice, questo ne ha determinato una difficoltà nella parte finale del lavoro.

Rileggendo il progetto sarebbe stato preferibile ideare interfacce specifiche invece che generiche. Quindi seguendo il principio ISP.